

Applied Human Computer Interaction (AHCI)

Asif Fazwani

Chapter 4 – Paradigms

What is a Paradigm?

A **widely accepted example or model** that shapes future designs.

Think of it like:

- TikTok's swipe-up feed → copied by Instagram Reels & YouTube Shorts
- iPhone's touch gestures → became industry standard

Human-Centered Innovation

The key idea: Great usability comes from enhancing human ability, not just machine power.

Technology	Creative Application	Result
AI	Chat interfaces like ChatGPT	Natural conversation
Camera	Snapchat filters	Fun self-expression
Touchscreen	Swipe gestures	Faster interaction

Paradigms Depend on Technology + Creativity

Most new interaction paradigms arise from:

- Technological breakthrough
- Creative design exploration
- User adoption
- Industry imitation

Important: New paradigms are often recognized **after** they succeed.

Example: Nobody predicted TikTok's short-video vertical swipe would dominate social media.

Paradigms Are Not Mutually Exclusive

Important concept: A single system may combine multiple paradigms.

Example: A modern smartphone includes:

- Graphical User Interface
- Touch Interaction
- Voice Interaction
- AI Assistance
- Cloud Computing

One device → Many paradigms

Why Study Paradigms?

Studying paradigms helps us:

- Understand evolution of interaction
- Recognize design innovation
- Predict future interaction trends
- Design better systems

Gaming Example: From keyboard games → motion control (Wii) → VR
→ AR

Each shift = New paradigm

Paradigm for Interaction: Time Sharing

What is Time Sharing?

Time Sharing = Multiple users using one computer simultaneously

Before time sharing:

- One user at a time
- Submit job → Wait → Get output
- After time sharing:
- Many users connected at once
- Interactive computing

This was the **birth of real Human-Computer Interaction**

Before Time Sharing – Batch Processing

How It Worked:

- Write program
- Convert to punch cards
- Submit to operator
- Wait hours/days
- Receive output

No immediate feedback

No interaction

No correction in real time

Imagine coding and waiting 24 hours just to see one error!

Paradigm for Interaction: Time Sharing

What Changed?

Instead of:  →  (One at a time)

We had:     →  (Shared simultaneously)

The computer:

- Switched rapidly between users
- Gave illusion of dedicated access
- Enabled live interaction

This was revolutionary.

Why Time-Sharing Matters in HCI?

Time sharing introduced:

- Real-time feedback
- Interactive programming
- Continuous human–computer dialogue
- User-centered computing

This marked the beginning of: True Human–Computer Interaction

Contemporary Analogy

Imagine if:

- Google Docs worked only once per day
- Instagram required submitting a “post request form”
- You had to wait 6 hours to see your TikTok draft

That’s batch processing.

Now imagine:

- Real-time comments
- Live multiplayer gaming
- Instant code compilation
- ChatGPT responding immediately

That’s time sharing.

The initial paradigm

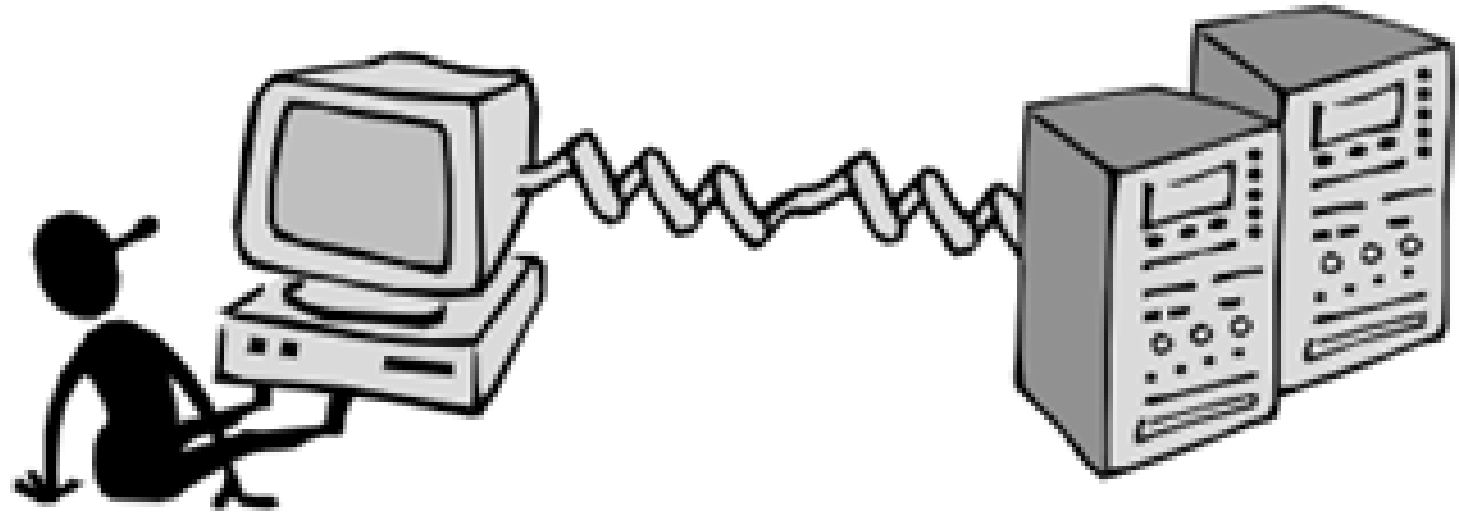
- Batch processing



Impersonal computing

Example Paradigm Shifts

- Batch processing
- Time-sharing



Interactive computing

Paradigm for Interaction: Video Display Units (VDUs)

Video Display Unit (VDU)

- A screen used to **display and manipulate computer information visually**

Before VDUs:

- Output = Printed paper
- Interaction = Text-based only

After VDUs:

- Output = Visual, dynamic, interactive

This marked the beginning of **visual interaction**

Paradigm for Interaction: Video Display Units (VDUs)

First Major Use: Military Systems

Semi-Automatic Ground Environment (SAGE)

- Used radar data
- Displayed strategic air defense information
- Allowed rapid decision-making

Why Visual? Strategic military decisions required:

- Fast understanding
- Pattern recognition
- Real-time updates
- Paper printouts were too slow.

Paradigm for Interaction: Video Display Units (VDUs)

The Breakthrough – 1962 (Ivan Sutherland MIT Graduate Student)

He developed: Sketchpad - *This changed computing forever*

What Was Sketchpad?

Sketchpad allowed users to:

- Draw directly on a screen
- Create visual models
- Store graphics in memory
- Manipulate objects interactively

Most importantly: Changing something on the screen changed the computer's memory.

This was revolutionary.

Contemporary Analogy

Imagine if:

- Photoshop had no preview
- Canva required typing coordinates
- Minecraft had no graphics
- Figma showed only code

That's pre-VDU computing.

Now imagine:

- Drag & drop design
- Touch-based drawing apps
- AR filters
- 3D modeling software

All descendants of Sketchpad.

Paradigm for Interaction: Programming Toolkits

Engelbart's Radical Thinking (1950s–60s)

- At that time, most people believed:

Computers = Complex machines

Only experts can use them

Engelbart believed:

Computers should teach and empower everyday humans.

This was revolutionary thinking.

Paradigm for Interaction: Programming Toolkits

Augmenting Human Intellect

Engelbart defined augmentation as Increasing a person's ability to:

- Solve complex problems
- Understand situations
- Develop solutions
- Combine intuition with powerful tools

Brain + Computer = Enhanced Intelligence

What is Programming Toolkit?

A toolkit provides:

- Reusable components
- Structured frameworks
- Built-in interaction patterns
- Faster development

Today's Examples:

- React (UI toolkit)
- Flutter
- Unity (game toolkit)
- AI APIs

Paradigm for Interaction: Programming Toolkits

1968, Engelbart presented: **oN-Line System**

Often called: “**The Mother of All Demos**”

He demonstrated: Mouse usage, Hypertext links, Real-time collaboration, Video conferencing

Why This Was a Paradigm Shift Engelbart’s team didn’t just build tools.

They built: **Programming toolkits**

Meaning:

- Systems that help build other systems
- Tools that expand user capability
- Platforms for innovation

Instead of writing everything from scratch, you use reusable components.

Contemporary Analogy

Imagine building an app:

- Without toolkit:
- Code everything manually
- Design every button from zero
- Create graphics engine from scratch

With toolkit:

- Drag & drop components
- Use prebuilt UI
- Plug into APIs
- Focus on creativity

Contemporary Time

From Engelbart to Today

- Engelbart's Vision → Toolkits → Platforms → Ecosystems

Modern Examples:

- App Store ecosystems
- Low-code/no-code platforms
- Figma design systems
- GitHub collaborative workflows

Today's dev culture = Descendant of Engelbart's philosophy

Paradigm for Interaction: Personal Computing

Earlier paradigms:

- Time sharing → programmers
- Toolkits → skilled developers

1970s shift:

- Computing power for the masses.

The goal: Make computers usable by **non-experts**.

This marks the beginning of **Personal Computing**

Paradigm for Interaction: Personal Computing

Making Computing Child-Friendly:

LOGO: Make programming accessible to children.

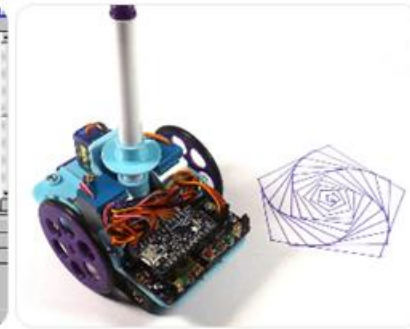
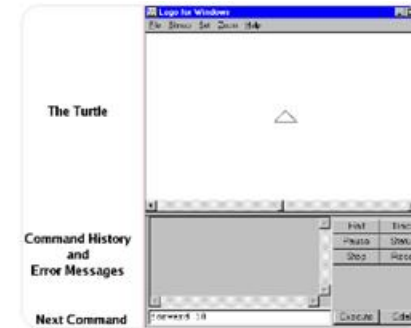
Children could type simple commands:

- Go forward → Turn left → Repeat 4

The turtle would draw shapes like:

- Squares → Circles → Patterns

Programming became visual and playful.



Paradigm for Interaction: Personal Computing

Why LOGO Was Revolutionary

- Before: Programming = Complex syntax + experts only
- After: Programming = Natural language + visual feedback

Papert proved: A powerful system becomes more powerful when it is easier to use.

Contemporary Example: Scratch, Minecraft Redstone, Roblox Studio

All inspired by this philosophy.

Paradigm for Interaction: Personal Computing

Alan Kay Inspired by: Engelbart's augmentation vision Papert's child-friendly programming Kay believed: **Computers should be personal, visual, and intuitive.**

Slide 6: Xerox PARC – The Innovation Hub Kay worked at: **Xerox Palo Alto Research Center (PARC)**

They developed: **Smalltalk** which introduced.

- Object-oriented programming
- Visual programming environments
- Window-based interfaces

Contemporary Time

Dynabook vision → Today's:

- iPad
- MacBook
- Surface
- Chromebooks
- Smartphones

Kay's 1970s idea = Your daily devices.

Even AI laptops and tablets reflect: Personal + Powerful + Interactive

Paradigm for Interaction: Windows System

Why Window Systems Were Needed

With personal computers becoming common:

One user

One machine

Many tasks at the same time

Problem:

Humans don't work linearly. We multitask.

Example: Writing assignment, Checking email, Watching tutorial,
Messaging friend

Computers needed to support this reality.

Paradigm for Interaction: Windows System

The Human Multitasking Problem

Humans:

- Switch between tasks
- Interrupt themselves
- Resume later
- Handle overlapping goals
- If a computer forces: Task A → Complete → Then Task B

That doesn't match human cognition.

We needed: Flexible dialogue, Context switching, Memory support

Paradigm for Interaction: Windows System

The Solution – Window = Separate visual workspace

Each window:

- Represents a task
- Shows a different context
- Maintains its own state

Windows help users:

- Remember where they left off
- Switch topics easily
- Manage multiple threads of activity

Paradigm for Interaction: Windows System

First Commercial WIMP System

In April 1981: **Xerox 8010 Star Information System**

Introduced:

- Windows
- Icons
- Mouse pointer
- Pull-down menus
- This system shaped modern computing.

Paradigm for Interaction: Windows System

Why Windows & WIMP Was a Paradigm Shift

It allowed:

- Task switching
- Visual task separation
- Direct manipulation
- User-controlled navigation

Instead of typing commands, users could:

Click, Drag, Select, Open

The interaction became spatial and visual.

Metaphor in HCI

A **metaphor** explains a new concept using something users already understand.

- Helps users quickly build a **mental model**.
- Common in interface design (windows, folders, buttons).
- Reduces fear for first-time users.

Simple Idea: "Make the digital world feel like the real world."

Example:



“Trash” icon = deleting files



“Folder” = organizing files



“Heart” on Instagram = liking something

When Metaphors Fail – Typewriter Problem

Early word processors used the **typewriter metaphor**.

Problem:

- On a typewriter → space is NOT a character.
- In a word processor → space IS a character.

Typists got confused.

Lesson: Metaphors help initially but can block deeper understanding.

Contemporary Example: Using the “phone call” icon  for voice chats in apps like Discord, but voice chat works very differently from traditional calls.

Cultural Problems with Metaphors

Not all cultures use the same office setup.

Some users may never have seen:

- Filing cabinets
- Floppy disks
- Desktop offices
- A metaphor that works in one country may fail in another.

Example:

- Floppy disk icon  still means “Save”
- But many Gen Z users have NEVER used a floppy disk!

Direct Manipulation

- Introduced by Ben Shneiderman (1982)
- Interaction through visible objects
- Users interact by clicking, dragging, touching
- Replaced complex command-line systems

Why It Emerged?

- Graphics hardware became cheaper (1980s)
- Need to make computers usable for non-programmers
- Shift from hackers → general public

Direct Manipulation

Command Line Interface (CLI)

- User types commands, No automatic feedback, Errors common (typos), Requires memorization

Example: mv file1.txt folder2/ → **Direct Manipulation GUI** Drag file into folder

Instant visual feedback, Hard to make syntax errors

Example: Compare: Using terminal on Mac vs Dragging files in Finder

Direct Manipulation

Key Features of Direct Manipulation

According to Shneiderman:

- Visibility of objects
- Incremental actions with rapid feedback
- Reversible actions (Undo!)
- Syntactic correctness
- Replace command language with object manipulation

Examples:

- Dragging Spotify songs into playlist
- Deleting WhatsApp message (“Delete for everyone”)

WYSIWYG Paradigm

WYSIWYG = What You See Is What You Get

- Screen representation $\approx$ Final output
- Minimal difference between preview & result
- Reduces guesswork

Examples:

- Canva designs
- Instagram post preview
- YouTube thumbnail editor

Activity 5

<https://tinyurl.com/Spring2026-Activity5>

Is a Word Processor a Direct Manipulation Interface?

1. Visibility of Objects

YES – Direct Manipulation

- Entire page visible
- Text formatting visible instantly
- Margins, spacing, fonts displayed
- Images shown where placed

We can see the visibility criterion for direct manipulation as very similar to the criteria for a WYSIWYG interface.

Is a Word Processor a Direct Manipulation Interface?

2. Incremental Actions with Rapid Feedback

YES – Direct Manipulation

- Typing shows instant characters
- Bold/Italic changes appear immediately
- Dragging image updates position in real-time
- Spellcheck underlines instantly

Is a Word Processor a Direct Manipulation Interface?

3. Reversibility of all actions, so that users are encouraged to explore without severe penalties YES – Direct Manipulation

- Undo (Ctrl + Z)
- Redo
- Version history (Google Docs)
- Track changes

This encourages experimentation.

Is a Word Processor a Direct Manipulation Interface?

4. Syntactic correctness of all actions, so that every user action is a legal operation - YES

WYSIWYG word processors usually provide menus and buttons which the user uses to articulate many commands. These interaction mechanisms serve to constrain the input language to allow only legal input from the user. Document markup systems, such as HTML and LaTeX, force the user to insert textual commands (which may be erroneously entered by the user) to achieve desired formatting effects.

Is a Word Processor a Direct Manipulation Interface?

5. Replacement of complex command languages with actions to manipulate directly the visible objects

The case for word processors is similar to that described above for syntactic correctness. In addition, operations on portions of text are achieved many times by allowing the user to highlight the text directly with a mouse (or arrow keys). Subsequent action on that text, such as moving it or copying it to somewhere else, can then be achieved more directly by allowing the user to 'drag' the selected text via the mouse to its new location.

Language vs Action Paradigm

What is the Action Paradigm?

- Users interact through **direct manipulation** (Clicking, dragging, swiping, touching)
- The interface becomes the *world itself*.

Examples: Dragging files into folders, Pinch-to-zoom on Instagram

What is the Language Paradigm?

- Users interact by **giving instructions using language**
- The interface acts as: A mediator, An agent. An interpreter
- Instead of doing, you *tell*.

Like old command-line interfaces.

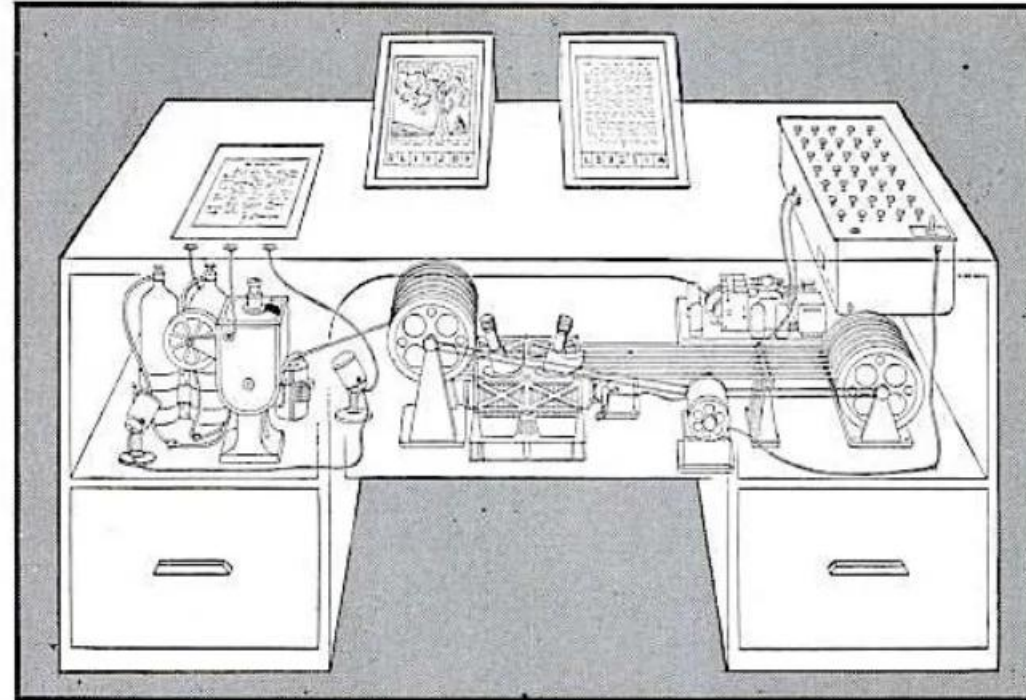
Example: `rm *.pdf` (remove pdf files)

Hypertext

Vannevar Bush proposed **Memex** vision

A desk-like machine that:

- Stored massive information
- Created associative links
- Mimicked how the human brain connects ideas



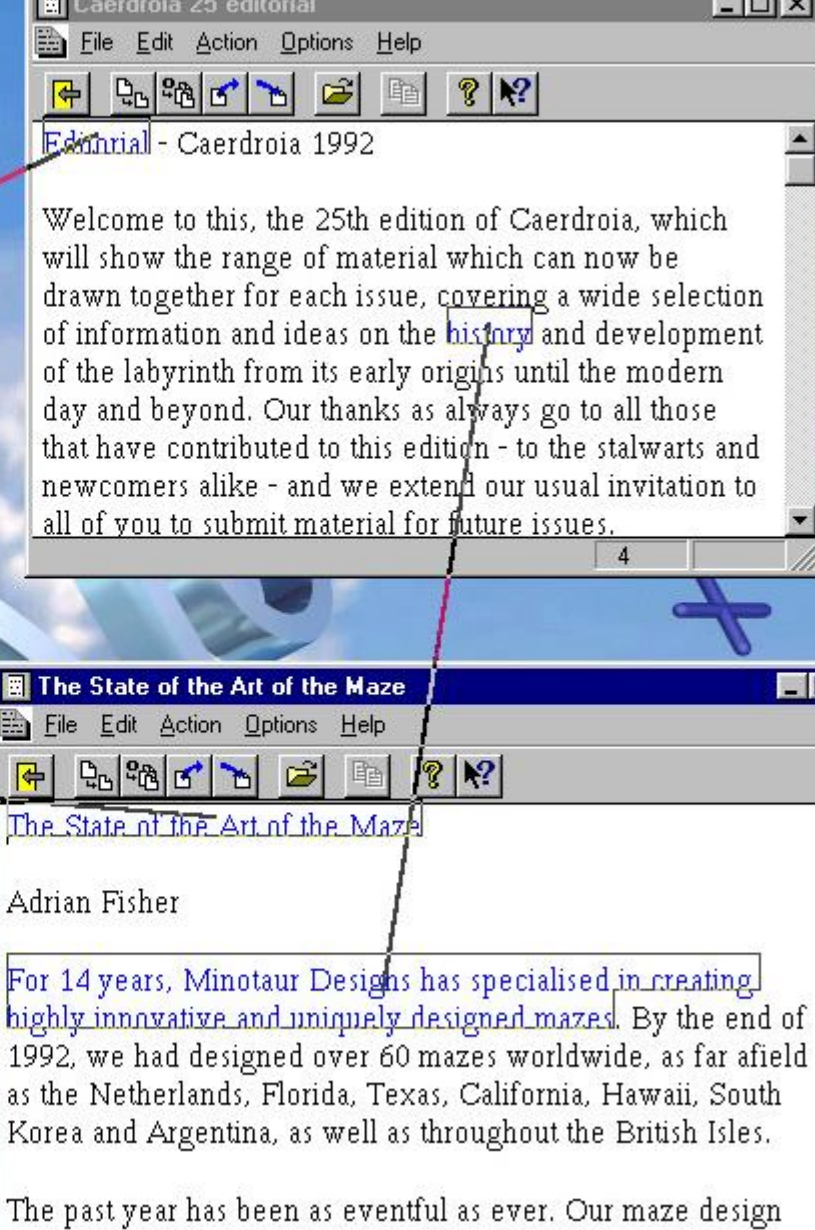
MEMEX in the form of a desk would instantly bring files and material on any subject to the operator's fingertips. Slanting translucent viewing screens magnify supermicrofilm filed by code numbers. At left is a mechanism which automatically photographs longhand notes, pictures and letters, then files them in the desk for future reference.

Why Memex Was Revolutionary?

Linear (start → end)

Memex: Non-linear
Information connected
through links.

- Brain connections
- Thought processes
- Idea networks



From Hypertext to Hypermedia

- Hypertext → Only text links
- Hypermedia → Links across: Text, Images, Audio, Video, Animation

Example:

YouTube video → Link → Shorts → Comments → Creator profile

TikTok “Part 2 in bio”

Instagram tagged posts

YouTube recommendations

Why Hypertext Matches Human Thinking

- Humans don’t think linearly.
- You think like: “Assignment → Professor → Previous lecture → That meme → Oh wait!”

Hypertext mirrors: Brain networks, Associative memory, Curiosity-driven learning

Why Hypertext Changed HCI Thoughts?

Before: Users consumed information passively.

After: Users Explore, Navigate, Control their path, Create links

Hypertext introduced: Interactive information

Multi-Modality

Multi-modality = Using **multiple human communication channels** to interact with a system. It emphasizes on simultaneous use of multiple channels for input and output

Each communication channel = a **modality**

Examples of human channels:

- Visual (seeing)
- Haptic (touch)
- Audio (speaking/hearing)

Example: Using **voice + touch + face recognition** to unlock and operate your smartphone.

Traditional vs Multi-Modal Systems

Traditional Interaction: Keyboard (input), Mouse (input), Screen (output), Basic sound (output)

Multi-Modal Interaction It combines Touch, Voice, Gesture, Facial expression, eye tracking, Sound feedback

Example Gaming with Controller (touch), Voice chat, Haptic vibration feedback

Question??

Is your phone truly multi-modal? Why or why not?

Possible Responses 1 - YES

It uses multiple input channels

- Touchscreen (tap, swipe, pinch)
- Voice commands (Siri/Google Assistant)
- Face recognition
- Fingerprint sensor
- Camera scanning QR codes

Example

- Unlock phone with Face ID
- Say Set alarm for 7 AM
- Swipe to check Instagram
- Phone vibrates for notification

Multiple human communication channels are used.

It uses multiple output channels

- Visual (screen display)
- Audio (notifications, voice assistant)
- Haptic feedback (vibration)
- Flash alerts for calls

So yes — phones clearly use multiple modalities.

Possible Responses 2 - NO

Because most actions are **not simultaneous integration** of channels.

Example:

- You either type OR speak
- You either tap OR use voice

The system does not always combine:

- Gesture + speech at the same time
- Eye tracking + voice + touch simultaneously

So it's multi-modal, but not deeply integrated.

Possible Responses 3 - It Depends on the Use Case

It depends on how you use it.

Example:

- Watching a reel → visual + audio
- Playing games → touch + haptic + audio
- Using AR filters → camera + touch + facial tracking

So, we can say

- Basic use → semi multi-modal
- Advanced apps (AR/AI) → genuinely multi-modal

Conclusion

Smartphones are multi-modal, but not perfectly integrated multi-modal systems. True multi-modality happens when systems combine multiple inputs simultaneously and intelligently like humans do.

Genuine Multi-Modal Systems

Not just multiple channels, but **simultaneous integration**.

Example:

You: Drag a file to trash and Say “Delete permanently”

System: Combines gesture + speech and Executes correct action

Example AI meeting tools:

- Analyze your voice tone
- Track facial expressions
- Generate meeting summary

Computer-Supported Cooperative Work (CSCW)

CSCW = **Computer-Supported Cooperative Work**

Systems that allow **people to collaborate through computers**

Focus shifts from “one user” → to “many users”

Not just interaction with computer, Interaction *through* computer

Example:

- Doing a group assignment on **Google Docs**
- Everyone edits at the same time
- Comments, suggestions, tagging teammates

The World Wide Web

The **World Wide Web (WWW)** is a system built on top of the Internet. It provides an **easy-to-use, graphical way** to access information. Hides technical complexity (IP addresses, protocols, servers). Users just click links, they don't see what happens behind the scenes.

Why the Web Matters in HCI The web changed interaction:

Before: Command-line interfaces, Technical knowledge required

After: Point-and-click browsing, Multimedia content, Anyone can publish, Web = Major shift in interaction paradigm

Agent-Based Interfaces

An **agent** is software that acts **on behalf of the user**

It can: Perform tasks, Monitor events, Make suggestions, Learn from behavior

Instead of you doing everything, The system helps or acts for you

Real-World Analogy

Human agents: Travel agents, Estate agents

Software agents: Email filters, Recommendation systems, AI assistants

Basic Agents (Rule-Based)

Some agents follow **simple rules**.

Example rule: IF sender = Bank, THEN mark email as High Priority

Real Example

- Filters in **Gmail**
- Spam detection
- Auto-sorting into Promotions/Social

Agents vs Direct Manipulation

Important HCI debate.

Direct Manipulation:

- User directly controls everything Example: Drag and drop files

Agent-Based Interface:

- Agent acts on your behalf
- You delegate control

Analogy:

- Traditional shopkeeper → brings items to you (agent)
- Supermarket assistant → helps while you choose (hybrid model)

Ubiquitous Computing

Ubiquitous computing = Computing everywhere

Computers embedded into everyday life

We don't go to a computer anymore

Coined and inspired by: **Mark Weiser** His famous idea:

“The most profound technologies are those that disappear.”

Core Idea Computers become:

- Invisible
- Embedded
- Seamless
- Part of daily life

Sensor-Based & Context-Aware Interaction

- Computers or environments sense user context automatically
- Use sensors to gather information about:
Environment, Online behavior, Body signals
- System infers what user wants or needs
- May act implicitly, without explicit instructions

Example:

Home Automation

- Lights turn on/off automatically
- Thermostat adjusts to your presence
- Smart fridges track groceries

Retail & E-Commerce

- Amazon recommends products based on purchase history
- Spotify / YouTube suggest content

Assistive Tech

- Automatic doors & washbasins
- Voice assistants helping elderly/disabled users

Implicit vs Explicit Interaction

Interaction Type	Description	Example
Explicit	User gives clear instructions to computer	Typing a search query
Implicit / Context-Aware	System predicts actions and acts automatically	Lights turning on when you enter a room

Context-aware computing = mostly implicit interaction